# Assignment 1 Handout

Scalable AI: Bridging Theory, Understanding, and Practice

EE 290 / 194 · Fall 2026

| | |
|---|---|
| **Due** | September 28, 2026 |
| **Where you work** | Course-provided H100 instance (8 × H100 per group) |
| **Grading split** | Part A (warm-up notebook): 40% \| Part B (MFU deep dive): 60% |
| **What you submit** | Completed notebook artifacts (Part A) + profiling + report (Part B). |
| **In-person check-in** | You will explain your Part A kernel code and present your Part B findings in person. |
| **How To Submit** | Please make sure to submit your assignment (notebooks, profiling results, report) on Gradescope. |

## Part A (40%): Triton kernel warm-up

**One notebook (`triton_kernel_warmup.ipynb`): write your own GPU kernels.**
Part A gets you set up on the course instance and comfortable writing, validating, and benchmarking your own Triton kernels. Implement the following four kernels:

(a) **Elementwise addition (forward, backward given):** $c = a + b$ for two $N$-element tensors.

(b) **RMSNorm (forward, backward):** $y_i = \dfrac{x_i}{\sqrt{\frac{1}{D}\sum_{j=1}^{D} x_j^2 + \epsilon}} w_i$ along the last dimension $D$.

(c) **SwiGLU (forward, backward):** $\mathrm{SwiGLU}(a, b) = \mathrm{SiLU}(a) \odot b$ where $\mathrm{SiLU}(a) = a\,\sigma(a)$. Fuse forward into one elementwise kernel and backward into another.

(d) **Cross-entropy (forward, backward):** fused log-softmax + NLL over logits $z \in \mathbb{R}^{B \times V}$. Forward computes per row $\ell_b = \log \sum_v e^{z_{bv} - m_b} - (z_{b,y_b} - m_b)$ with the max-subtraction trick. Backward computes $\partial \ell / \partial z = \frac{1}{B}\left(\mathrm{softmax}(z) - \mathbf{1}_y\right)$ without materializing the full $(B, V)$ probability matrix in HBM.

The notebook has the eager references, correctness tests, and benchmarks for each kernel, along with a checklist of what to report. Every check must pass in your submitted run and the results table must be filled in. Use the skeleton from kernel (a) as a template for the rest.

**In-person check-in:** you will be asked to explain your kernel code during an in-person check-in, so make sure you can walk through your implementations and justify your choices.

## Part B (60%): MFU on real training workloads

Part B involves an open-ended systems investigation on real training/SFT workloads. You will measure where performance goes, diagnose the bottlenecks, and implement interventions to improve it.

**Your job:**

- Choose a training/SFT stack and run workloads end-to-end. For example, you may start from NeMo AutoModel or NeMoRL with any backend (DTensor-based or Megatron-Core-based).
- Please select Nemotron-Nano-V3 as your target model.
- If your framework does not support Nemotron-Nano-V3, make sure you integrate support for it. If it does, you can use the existing support and improve upon it.
- For the framework you have chosen, measure various efficiency metrics (throughput and MFU, etc.), then profile and explain where performance is being lost.

- Diagnose bottlenecks with evidence, then design and implement interventions (anything reasonable, including changing the parallelism strategy, writing kernels, reimplementing abstractions, etc.). You are welcome to go as grand as you want.
- If you improve performance and MFU over the baseline behavior in the repo/framework, we encourage you to open a PR with reproduction steps. This is optional and not part of your grade. If the improvement is meaningful and reproducible, we will help publicize it.

**In-person check-in:** you will present your Part B findings during the in-person check-in.

## What we care about

Your report must explain the path from observation to fix. What was the symptom, what did you conclude the bottleneck was and on what evidence, what did you change, and what was the measured effect. Your report should defend why you believed something was the bottleneck, what evidence supported that belief, and why your change should help from first principles. Generally, had you *only* done an exhaustive random search, you probably would not be able to write a good report here.

## Deliverables

- **Part A:** required notebook artifacts per the notebook checklists.
- **Part B:** baseline and improved commands/configs, plus scripts/notebooks used for measurement.
- **Profiling evidence:** traces/summaries that support your diagnosis, **including Nsight screenshots in the report**. Include enough context to reproduce.
- **Pull request (optional, not graded):** before/after MFU numbers and clear reproduction steps.
- **Report (PDF).**

## Required report structure

The report should be 1–4 pages plus an appendix if needed.

1. Setup + exact reproduction steps for baseline.
2. Baseline MFU/throughput and what "current OSS best" looks like.
3. Profiling results: **Nsight screenshots** of the key traces and what they imply.
4. Diagnosis: a few evidence-backed hypotheses.
5. Interventions: what you changed and why it should help.
6. Results: before/after + good ablations explaining why it helps.
7. Reproducibility notes: seeds, caveats, end-to-end run.

## Grading rubric (100%)

| Component | Weight |
|---|---|
| Part A: Triton kernel notebook | 40% |
| Part B: Measurement setup + baseline reproduction | 10% |
| Part B: Diagnosis with profiling evidence | 20% |
| Part B: Intervention engineering + reproducibility | 20% |
| Part B: Report | 10% |

## Additional notes on Negative Results

You can earn full credit without "winning" the MFU wars, but you cannot earn full credit with ungrounded random search. Even if you do not improve performance, you can still earn full credit for a good diagnosis and interventions that you attempted, even if they did not help improve over baseline. What matters is that your experiments were systematic and your reasoning is clear.